Bézier Curve — Matrix Method

$$P(u) = U \cdot M \cdot G$$

Explanation of the Program Output

Case 2: Quartic 3-D, 5 Control Points

`03-bezier-matrix-pedagogical.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains the output of `03-bezier-matrix-pedagogical.jl` when run with `CASE = 2`. The program computes the same quartic 3-D Bézier curve as the previous scripts, but using an entirely different mathematical method: the **matrix method**. Instead of expanding Bernstein polynomials algebraically, the curve is expressed as a single matrix product $P(u) = U \cdot M \cdot G$, which is then evaluated by substituting powers of u into a row vector.

Contents

1 The Big Idea: One Formula, Three Matrices **2**

2 Header and Control Points **2**

3 Step 1 — Matrix Formulation **3**

4 Step 2 — Deriving the Basis Matrix M **3**

4.1 What M encodes . . . 3

4.2 The formula for each entry . . . 4

4.3 Column-by-column derivation . . . 4

4.4 The full M matrix . . . 5

5 Step 3 — Computing $M \cdot G$ **5**

6 Step 4 — Evaluating $U \cdot (M \cdot G)$ **6**
6.1 Structure of each block 6
6.2 Reading the U vector 6
6.3 Reading the dot product lines 6
6.4 The boundary cases 7

7 Step 5 — Summary Table **8**

8 Why the Matrix Method? **8**

1. The Big Idea: One Formula, Three Matrices

All previous scripts expanded Bernstein polynomials term by term. This script takes a different route: it packages the entire curve into a single matrix equation and evaluates it with one matrix multiplication.

The entire Bézier curve is encoded in three matrices:

$$P(u) = U(u) \cdot M \cdot G$$

Matrix	Shape	Role
$U(u)$	1×5	**Parameter vector.** A row of powers of u: $[u^4,\ u^3,\ u^2,\ u,\ 1]$. This is the only part that changes when you evaluate at different u values.
M	5×5	**Basis matrix.** Encodes all the Bernstein expansion coefficients. Computed once from the degree and never changes.
G	5×3	**Geometry matrix.** The control point coordinates, one row per point, one column per axis (x, y, z).

Because M and G do not depend on u, the program computes $M \cdot G$ *once* and stores it. Evaluating the curve at any u then only requires building $U(u)$ and computing one dot product.

> **Why this matters.** With five evaluation points and three axes, the naïve approach calls `evaluate()` fifteen times (five u values $\times$ three axes). The matrix method batches all evaluations into a single matrix multiplication $U_{\text{all}} \cdot (M \cdot G)$ where U_{all} is a 5×5 matrix stacking all $U(u)$ rows. For large numbers of u values this is dramatically faster.

2. Header and Control Points

```
=================================================================
  Bezier Curve - Matrix Method  [P(u) = U * M * G]
  Case 2  |  Quartic  |  3D  |  5 Control Points
=================================================================

  Control Points:
    P0 :  x = 0  y = 0  z = 1
    P1 :  x = 0  y = 4  z = 1
    P2 :  x = 4  y = 0  z = 1
    P3 :  x = 4  y = 4  z = 1
    P4 :  x = 5  y = 4  z = 1
```

The subtitle `[P(u) = U * M * G]` states the method upfront. The control points are the same five points used in the previous scripts. They will form the geometry matrix G.

3. Step 1 — Matrix Formulation

```
STEP 1 - Matrix Formulation:  P(u) = U * M * G
----------------------------------------------------------------

P(u)  =  U(u)  *  M  *  G

U(u)  shape (1x5)  parameter vector  [u^4, ..., u, 1]
M     shape (5x5)  basis matrix  (derived from Bernstein)
G     shape (5x3)  geometry matrix  (control points)

    x  y  z
    -  -  -
P0  0  0  1
P1  0  4  1
P2  4  0  1
P3  4  4  1
P4  5  4  1
```

This step defines the three matrices and displays G — the geometry matrix — directly as a table of control points.

The geometry matrix G is simply the control points written as rows:

$$G = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 4 & 1 \\ 4 & 0 & 1 \\ 4 & 4 & 1 \\ 5 & 4 & 1 \end{bmatrix}$$

Each row is one control point; each column is one axis. The table in the output is exactly this matrix, labelled with axis names and point names.

> **Row and column ordering.** Rows go $P_0, P_1, \ldots, P_4$ (top to bottom). Columns go x, y, z (left to right). This ordering is consistent throughout the output: $G[i, j]$ is the j-th coordinate of the i-th control point.

4. Step 2 — Deriving the Basis Matrix M

This is the most mathematical step. It is also the heart of the matrix method. Take your time here — once you understand M, the rest is just arithmetic.

4.1. What M encodes

```
Each column i of M holds the monomial coefficients
of the Bernstein basis polynomial B(4, i, u):

  B(n,i,u)  =  C(n,i) * u^i * (1-u)^(n-i)

Expanding (1-u)^(n-i) via the binomial theorem:
  (1-u)^m  =  sum  C(m,j) * (-1)^j * u^j

Coefficient of u^k in B(n,i,u):
  M[n-k, i]  =  C(n,i) * C(n-i, k-i) * (-1)^(k-i)   for k >= i, else 0

Row ordering: row 0 = u^4 (highest),  row 4 = u^0 (constant)
```

Each column i of M contains the monomial coefficients of $B(4, i, u)$, reading from the highest power down to the constant.

The row index encodes the power of u: row 0 holds the coefficient of u^4, row 1 holds the coefficient of u^3, ..., row 4 holds the constant term (u^0).

Why store coefficients this way?

When we compute $U \cdot M$, each element of the result is a dot product of the row vector $[u^4, u^3, u^2, u, 1]$ with one column of M. If column i of M holds the coefficients of $B(4, i, u)$ from highest to lowest power, then the dot product $U \cdot M_{\text{col } i}$ automatically evaluates $B(4, i, u)$ at the given u — because it computes exactly $u^4 \cdot c_4 + u^3 \cdot c_3 + u^2 \cdot c_2 + u \cdot c_1 + 1 \cdot c_0$.

4.2. The formula for each entry

The formula shown in the output:

$$M[n-k,\ i] = \binom{n}{i}\binom{n-i}{k-i}(-1)^{k-i} \quad \text{for } k \geq i, \text{ else } 0$$

This comes directly from expanding $B(n, i, u) = \binom{n}{i} u^i (1-u)^{n-i}$ using the binomial theorem on $(1-u)^{n-i}$ and collecting the coefficient of u^k.

4.3. Column-by-column derivation

```
Column 0  ->  B(4,0,u)  =  (1-u)^4
  M[0,0]  =     1   (coeff of u^4)
  M[1,0]  =    -4   (coeff of u^3)
  M[2,0]  =     6   (coeff of u^2)
  M[3,0]  =    -4   (coeff of u)
  M[4,0]  =     1   (coeff of 1)

Column 1  ->  B(4,1,u)  =  4u(1-u)^3
  M[0,1]  =    -4   (coeff of u^4)
  M[1,1]  =    12   (coeff of u^3)
  M[2,1]  =   -12   (coeff of u^2)
```

```
M[3,1]  =     4   (coeff of u)
...
```

Each column is shown with its non-zero entries only. Let us verify column 0 by hand.

Verify column 0: $B(4, 0, u) = (1 - u)^4$

Expand $(1 - u)^4$ using the binomial theorem:

$$(1 - u)^4 = \sum_{j=0}^{4} \binom{4}{j} (-1)^j u^j = 1 - 4u + 6u^2 - 4u^3 + u^4$$

Reading coefficients from highest power down:

$$u^4 \to +1, \quad u^3 \to -4, \quad u^2 \to +6, \quad u^1 \to -4, \quad u^0 \to +1$$

These are exactly $M[0, 0] = 1$, $M[1, 0] = -4$, $M[2, 0] = 6$, $M[3, 0] = -4$, $M[4, 0] = 1$. ✓

Verify column 2: $B(4, 2, u) = 6u^2(1 - u)^2$

$(1 - u)^2 = 1 - 2u + u^2$, so:

$$6u^2(1 - 2u + u^2) = 6u^2 - 12u^3 + 6u^4$$

Only three non-zero entries (the u^0 and u^1 terms are zero):

$$u^4 \to +6, \quad u^3 \to -12, \quad u^2 \to +6$$

So $M[0, 2] = 6$, $M[1, 2] = -12$, $M[2, 2] = 6$, and all lower rows in column 2 are zero. ✓

4.4. The full M matrix

```
Full M matrix:

     P0   P1   P2  P3  P4
     --  ---  ---  --  --
u^4   1   -4    6  -4   1
u^3  -4   12  -12   4   0
u^2   6  -12    6   0   0
u^1  -4    4    0   0   0
u^0   1    0    0   0   0
```

Written as a matrix (rows = powers from u^4 down, columns = B_0 through B_4):

$$M = \begin{bmatrix} 1 & -4 & 6 & -4 & 1 \\ -4 & 12 & -12 & 4 & 0 \\ 6 & -12 & 6 & 0 & 0 \\ -4 & 4 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Pattern in M. The lower-left triangle is all zeros — this is because $B(4, i, u)$ contains u^i as its lowest power, so all powers below u^i in column i are zero. The matrix is therefore **upper-triangular** (in the sense that all entries below the anti-diagonal are zero).

The first column contains the coefficients of $(1-u)^4$, which is the familiar $1, -4, 6, -4, 1$ pattern from Pascal's triangle with alternating signs. The last column contains a single 1 in the top row — because $B(4, 4, u) = u^4$ has only one term.

5. Step 3 — Computing $M \cdot G$

```
STEP 3 - Compute  M * G  - Polynomial Coefficient Matrix
----------------------------------------------------------------

M * G  gives the monomial coefficients for each axis.
Row k of (M*G) is the coefficient of u^(n-k).

       x    y  z
     ---  ---  -
u^4   13  -28  0
u^3  -32   64  0
u^2   24  -48  0
u^1    0   16  0
u^0    0    0  1

Resulting polynomials:
  x(u)  =  13u^4  - 32u^3  + 24u^2
  y(u)  =  -28u^4  + 64u^3  - 48u^2  + 16u
  z(u)  =  1
```

$M \cdot G$ is a 5×3 matrix. Each row gives the coefficient of one power of u for all three axes simultaneously. Row 0 (labelled u^4) gives the leading coefficients; row 4 (labelled u^0) gives the constant terms.

Verify the x column of $M \cdot G$:

The x column of G is $[0, 0, 4, 4, 5]^T$. Multiplying $M \cdot [0, 0, 4, 4, 5]^T$ row by row:

Row 0 (coeff of u^4):

$$1(0) + (-4)(0) + 6(4) + (-4)(4) + 1(5) = 0 + 0 + 24 - 16 + 5 = 13\checkmark$$

Row 1 (coeff of u^3):

$$(-4)(0) + 12(0) + (-12)(4) + 4(4) + 0(5) = 0 + 0 - 48 + 16 + 0 = -32\checkmark$$

Row 2 (coeff of u^2):

$$6(0) + (-12)(0) + 6(4) + 0(4) + 0(5) = 0 + 0 + 24 + 0 + 0 = 24\checkmark$$

Rows 3 and 4 (coeff of u and constant):

$$(-4)(0) + 4(0) + 0(4) + 0(4) + 0(5) = 0\checkmark$$

$$1(0) + 0(0) + 0(4) + 0(4) + 0(5) = 0\checkmark$$

So the x column of $M \cdot G$ is $[13, -32, 24, 0, 0]^T$, giving $x(u) = 13u^4 - 32u^3 + 24u^2$. ✓

Verify the z column of $M \cdot G$:

The z column of G is $[1, 1, 1, 1, 1]^T$. Each row of M sums to zero except the constant row:

The columns of M are the monomial expansions of $B(4, 0)$ through $B(4, 4)$. Their sum is the expansion of $\sum_{i=0}^{4} B(4, i, u) = 1$, which in monomial form is simply $u^0 = 1$ — all higher powers cancel.

Therefore $M \cdot [1, 1, 1, 1, 1]^T = [0, 0, 0, 0, 1]^T$, giving $z(u) = 1$. ✓

6. Step 4 — Evaluating $U \cdot (M \cdot G)$

This step is the most explicit in the entire output. For each u value, the program shows the full U vector and then writes out every term of every dot product. Nothing is hidden.

6.1. Structure of each block

Each u value gets its own block with two parts: the U vector table, then the dot product expansion.

```
-- u = 0.5 ------------------------------------------------

U  =
  u^4   u^3   u^2   u^1  u^0
-----  ----  ----  ----  ---
1//16  1//8  1//4  1//2    1

U * (M*G)  =  P(u):
  x(u)  =  1//16*(13)  +  1//8*(-32)  +  1//4*(24)
           +  1//2*(0)  +  1*(0)
         =  45//16
  y(u)  =  ...
```

The U vector is displayed as a single-row table with each power of u in its own column. The values are shown as exact fractions (`1//16`, `1//8`, etc.) because the program uses `fmt_frac` which converts floats back to their nearest rational representation.

6.2. Reading the U vector

The U vector for $u = \frac{1}{2}$ is:

$$U = \left[\left(\tfrac{1}{2}\right)^4,\ \left(\tfrac{1}{2}\right)^3,\ \left(\tfrac{1}{2}\right)^2,\ \tfrac{1}{2},\ 1\right] = \left[\tfrac{1}{16},\ \tfrac{1}{8},\ \tfrac{1}{4},\ \tfrac{1}{2},\ 1\right]$$

These are exactly the values shown in the output table: `1//16`, `1//8`, `1//4`, `1//2`, `1`.

6.3. Reading the dot product lines

Each axis line shows:

$$\text{axis}(u) = U[0] \cdot (MG)[0] + U[1] \cdot (MG)[1] + \cdots + U[4] \cdot (MG)[4]$$

where $(MG)[k]$ is the entry from the pre-computed $M \cdot G$ column for that axis, at row k.

Hand-verify $x\left(\frac{1}{2}\right) = \dfrac{45}{16}$:

From Step 3, the x column of $M \cdot G$ is $[13, -32, 24, 0, 0]$. The U vector at $u = \frac{1}{2}$ is $[\frac{1}{16}, \frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1]$.

Dot product:

$$\frac{1}{16} \cdot 13 + \frac{1}{8} \cdot (-32) + \frac{1}{4} \cdot 24 + \frac{1}{2} \cdot 0 + 1 \cdot 0$$
$$= \frac{13}{16} - \frac{32}{8} + \frac{24}{4} = \frac{13}{16} - 4 + 6 = \frac{13}{16} + 2 = \frac{13+32}{16} = \frac{45}{16}$$

Displayed as `45//16` $= 2.8125$. ✓

Hand-verify $y\left(\frac{1}{4}\right) = \dfrac{121}{64}$:

From Step 3, the y column of $M \cdot G$ is $[-28, 64, -48, 16, 0]$. The U vector at $u = \frac{1}{4}$ is $[\frac{1}{256}, \frac{1}{64}, \frac{1}{16}, \frac{1}{4}, 1]$.

Dot product (converting to denominator 256):

$$\frac{1}{256}(-28) + \frac{1}{64}(64) + \frac{1}{16}(-48) + \frac{1}{4}(16) + 1(0)$$

$$= \frac{-28}{256} + \frac{256}{256} + \frac{-768}{256} + \frac{1024}{256} = \frac{-28+256-768+1024}{256} = \frac{484}{256} = \frac{121}{64}$$

Displayed as `121//64` ≈ 1.8906. ✓

6.4. The boundary cases

```
-- u = 0.0 ------------------------------------------------
U  =  [0  0  0  0  1]

-- u = 1.0 ------------------------------------------------
U  =  [1  1  1  1  1]
```

At $u = 0$: $U = [0, 0, 0, 0, 1]$.

The dot product with any column of $M \cdot G$ picks only the last row — the constant term (row 4, u^0).

For x: the constant term is 0. Result: $x(0) = 0$. For z: the constant term is 1. Result: $z(0) = 1$.

So $P(0) = (0, 0, 1) = P_0$. ✓The curve starts at the first control point.

At $u = 1$: $U = [1, 1, 1, 1, 1]$.

The dot product sums all rows of the $M \cdot G$ column.

For x: $13 + (-32) + 24 + 0 + 0 = 5$. ✓For y: $-28 + 64 + (-48) + 16 + 0 = 4$. ✓

So $P(1) = (5, 4, 1) = P_4$. ✓The curve ends at the last control point.

7. Step 5 — Summary Table

```
STEP 5 - Summary - Evaluation Table
-----------------------------------------------------------------

+=========================================================+
| u       x(u)                y(u)              z(u)       |
|=========================================================|
| 0       0  (0.0000)         0  (0.0000)  1  (1.0000)     |
| 0.2500  269//256  (1.0508) 121//64 (1.8906) 1 (1.0000) |
| 0.5000  45//16  (2.8125)   9//4  (2.2500)  1  (1.0000) |
| 0.7500  1053//256  (4.1133)201//64(3.1406)  1  (1.0000) |
```

```
| 1       5  (5.0000)        4  (4.0000)  1  (1.0000)    |
+==========================================================+
```

The final summary table collects all results. It uses a box-border style (with Unicode box-drawing characters `+`, `|`, `=`) to visually distinguish it from the plain tables used in earlier steps.

Each cell shows the exact rational result followed by the 4-decimal-place float in parentheses: `269//256 (1.0508)` means the exact value is $\frac{269}{256}$ and the decimal approximation is 1.0508.

All results match the previous scripts. The five points on the curve are exactly the same as those computed by the Bernstein expansion in scripts 01 and 02. This is the expected result: the matrix method and the algebraic expansion are mathematically equivalent. They arrive at the same answer by different routes.

$u = 0$: $(0, 0, 1) = P_0$. The curve starts at P_0.
$u = 1$: $(5, 4, 1) = P_4$. The curve ends at P_4.
$z(u) = 1$ for all u: the curve lies flat in the plane $z = 1$.

8. Why the Matrix Method?

The matrix method produces the same curve. So why use it at all? Three reasons.

Efficiency. $M \cdot G$ is computed once. Evaluating the curve at N different u values then costs only one $N \times (n + 1)$ matrix multiplication. Compare this to expanding and evaluating $n + 1$ Bernstein polynomials separately for each of N values.

Insight. The matrix M is fixed for any curve of the same degree. Only G changes when you move control points. This makes it easy to see how changing one control point affects the coefficients: you only need to update one row of G and recompute $M \cdot G$.

Transparency. Step 4 in this output is the most explicit evaluation shown in any of the three scripts. Every multiplication is visible. There is no recursion, no accumulation — just a row vector times a pre-computed matrix.

Summary of the computation chain:

Step	Operation
Once	$M \leftarrow$ derive_M(degree)
Once	$MG \leftarrow M \cdot G$
Per u	$U \leftarrow [u^4, u^3, u^2, u, 1]$
Per u	$P(u) \leftarrow U \cdot MG$

The heavy work (M derivation and $M \cdot G$ multiplication) is done once. All repeated evaluations are cheap.

Output explanation for `03-bezier-matrix-pedagogical.jl`, Case 2 | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated